1. 增强代理抗封锁能力 3. 功能扩展批量节点导入，批量节点探活检测，批量节点地区检测，节点探活检测次数与成功率最后给节点打标签越健康的节点分数越高；节点按国家分组；还有分流功能，可以自定义给某些软

https://gemini.google.com/app/02cdb795a6187d0b

User prompt: 1. 增强代理抗封锁能力 3. 功能扩展批量节点导入，批量节点探活检测，批量节点地区检测，节点探活检测次数与成功率最后给节点打标签越健康的节点分数越高；节点按国家分组；还有分流功能，可以自定义给某些软件走哪些国家，另一些走另外的国家，未设置的走默认国家；节点不可用自动切换（切换的逻辑需要详细讨论，或者支持多种方案，并且可以由用户自定义自动切换方案,提前设置一些大的方向，细节用户把控）；支持节点订阅；支持将节点设为收藏；你也提一下实用的功能 4. 耗电优化，尽量节省电量。 5. 优化程序代码逻辑，尽可能缩小安装包大小。

Response: 针对 **SingFlow**（基于 `libbox.aar` / `sing-box` 内核的 Android 客户端）的演进需求，以下从抗封锁能力、核心功能扩展（含健康度评分、应用级国家分流、自定义容灾切换等）、功耗控制以及安装包瘦身四个维度提供完整的技术架构与实现方案。

一、 增强代理抗封锁与传输抗干扰能力

在基于 `sing-box` (`libbox.aar`) 的客户端中，抗封锁能力主要依赖于**传输层伪装、握手指纹混淆、多路复用以及防 DNS 污染**：

1. **主流抗封锁协议与现代传输层支持**
   - **VLESS + XTLS-Vision / Reality**：在现有 VLESS 支持的基础上，完整适配 `REALITY` 协议，消除服务端 TLS 证书特征，直接借用知名站点的真实证书与指纹，抵御主动探测与 SNI 阻断。
   - **基于 QUIC 的抗丢包协议（Hysteria 2 / TUIC v5）**：针对劣化网络与高丢包环境，引入基于 UDP/QUIC 的协议，利用改进的拥塞控制算法提升弱网吞吐量，并启用端口跳跃（Port Hopping）规避单端口 UDP QoS 限流。
   - **WebSocket / gRPC / HTTPUpgrade + CDN 兜底**：支持将流量伪装为标准 Web 流量，配合 TLS 与 CDN 中转，作为高封锁时期的保底通道。
2. **TLS 客户端指纹伪装（uTLS）与抗分片检测**
   - **uTLS 指纹模拟**：在 outbound 的 `tls.utls` 配置中默认开启 `chrome`、`firefox` 或 `safari` 随机/指定指纹模拟，消除 Go 语言原生 TLS 握手特征（JA3/JA4 指纹）。
   - **TLS Hello 分片（TLS Fragment）**：在 `sing-box` 路由或拨号配置中支持 `tls_fragment`（或通过底层 dialer 参数控制分段发送 ClientHello 包及延迟时间），阻断中间设备对 SNI 明文字段的深度包检测（DPI）。
3. **多路复用与填充（Multiplex & Padding）**
   - 集成 `sing-mux`（支持 `h2mux` / `smux` / `yamux`），开启 `padding: true`（流量长度填充），模糊特定握手包的长度特征，同时减少频繁建连带来的握手开销与暴露风险。
4. **防泄漏安全 DNS 架构（Anti-DNS Escape）**
   - 针对真实节点测试中容易出现的 DNS 逃逸问题，弃用明文 UDP 53 查询，全面采用 **DoH (DNS over HTTPS) / DoT / DoQ**。
   - 开启 `FakeIP` 模式或严格的远端解析（Proxy DNS）：境内域名通过本地 DoH（如阿里/腾讯 DoH）直连解析，境外/未知域名强制通过代理隧道走远端 DoH（如 `[https://1.1.1.1/dns-query](https://1.1.1.1/dns-query)`）解析，彻底杜绝 DNS 抢答与污染。

二、 核心功能扩展与架构设计

1. 批量节点导入、订阅管理与收藏机制

- **多格式解析器**：支持剪贴板批量多行导入（`vless://`、`vmess://`、`ss://`、`trojan://`、`hysteria2://`、`tuic://`）、Base64 订阅内容解析、Clash YAML 以及 `sing-box` 原生 JSON 格式导入，并统一处理节点名称 URL 编码/特殊字符解码问题。
- **订阅管理引擎**：
  - 支持配置多个订阅源 URL、自定义 `User-Agent`、定时后台静默更新（基于 Android `WorkManager`）。
  - 解析订阅响应头中的 `subscription-userinfo`（提取 `upload`、`download`、`total`、`expire`），在 UI 卡片展示剩余流量与到期时间。
- **节点去重与收藏（Favorites）**：

- 以 (protocol, server, server_port, uuid/password, transport_type) 生成唯一 Hash 指纹实现导入自动去重。
- 在本地 Room 数据库的 NodeEntity 表中增加 isFavorite: Boolean 与 customOrder: Int 字段；订阅更新覆盖时，通过 Hash 匹配保留用户的“收藏”状态与历史健康分。

2. 批量探活、地区检测与健康度评分模型

- **高并发探活检测：**
  - 利用 libbox 内置的 URLTest 接口或协程并发池（限制并发度如 Semaphore(15)，避免瞬间耗尽本地 Socket 或触发路由器断流），对目标地址（默认 https://www.gstatic.com/generate_204，支持用户自定义）发起真实代理握手并记录 RTT 延迟。
- **批量节点落地地区检测与国家分组：**
  - 探活成功后，复用该代理通道请求轻量级 GeoIP 接口（如 https://www.cloudflare.com/cdn-cgi/trace 或 https://api.ip.sb/geoip），提取落地 IP 对应的两字母国家代码（ISO 3166-1 alpha-2，如 US、JP、SG、HK）。
  - 数据库按 countryCode 自动归类建组，并在 sing-box 配置生成时映射为对应国家的 Outbound Group（如 group-US、group-JP）。
- **节点健康度评分与动态标签算法：**
  - **持久化指标：**每个节点记录总检测次数 N、成功次数 S、连续失败次数 F_{streak}、指数移动平均延迟 L_{ema}（毫秒）以及延迟抖动 σ_L。
  - **平滑成功率（拉普拉斯平滑）：**防止仅测 1 次成功就拿到满分，采用平滑公式：

$$R = \frac{S+1}{N+2}$$

  - **综合健康分（0–100 分）：**

$$\text{Score} = \text{clamp}\left(R \times 60 + \max\left(0, 1 - \frac{L_{ema}}{1500}\right) \times 30 + \min(10, \ln(N+1) \times 3) - F_{streak} \times 12,\ 0,\ 100\right)$$

  - **自动打标签：**
    - Score >= 85：**极佳 (专线级)**
    - 70 <= Score < 85：**健康 (稳定)**
    - 50 <= Score < 70：**一般 (偶有波动)**
    - Score < 50 或 $F_{streak} \geq 3$：**异常/高危 (建议淘汰)**

3. 按应用自定义国家分流（Per-App Country Routing）

利用 sing-box 强大的 route.rules 引擎，无需开启多个 VPN 实例即可实现不同 App 走不同国家节点：

路由层级	匹配条件 (route.rules)	目标出站 (outbound)	作用说明
系统与局域网直连	ip_is_private: true	direct	保障局域网投屏、打印机与内网通信正常
自定义应用分流	package_name: ["com.netflix.mediaclient", ...]	group-US (美国节点组)	用户指定的软件强制走指定国家出站组
多国自定义映射	package_name: ["jp.naver.line.android", ...]	group-JP (日本节点组)	另一批软件走另一国家出站组
国内应用/域名分流	rule_set: ["geosite-cn", "geoip-cn"]	direct	微信、淘宝等国内软件及国内 IP 直连（可选开关）

路由层级	匹配条件 (route.rules)	目标出站 (outbound)	作用说明
默认兜底路由	route.final	group-DEFAULT	未单独设置的应用统一走用户设定的默认国家组或全局最优组

- **实现方式**：在客户端提供“应用分流”可视化配置页，读取已安装应用列表（含应用图标、名称、包名），允许用户将任意 App 绑定到“直连”、“阻断”或特定“国家分组/收藏分组”。启动 VPN 时，动态编译生成上述 route.rules JSON 注入 libbox。

4. 节点不可用自动切换方案（分层可配置容灾架构）

将自动切换逻辑拆分为“预设大方向模式”**与**“用户微调参数”两层，结合 sing-box 的 urltest / selector 出站与客户端健康分状态机协同控制：

- **预设四大切换方向（用户一键选择）**：
  1. **同国家优先容灾（适合流媒体/固定 IP 归属地需求）**：当前节点断连时，仅在**同一国家分组**内按“健康度最高 -> 延迟最低”顺序平替；仅当该国所有节点全军覆没时，才弹窗提示或切往备用国家。
  2. **极速低延迟优先（适合游戏/高频交互）**：跨国家边界，在“收藏节点”或“高健康分节点池（Score >= 75）”中实时选择 RTT 最低的节点；一旦延迟飙升超过阈值立即切换。
  3. **高可用稳健优先（适合日常挂机/长连接）**：引入迟滞防抖（Hysteresis）机制，只要当前节点可用且延迟未恶化至不可接受程度，即使其它节点延迟低几十毫秒也不触发切换，避免频繁切换导致 TCP 连接重置或账号风控。
  4. **手动锁定 + 断网保底**：平时固定使用用户手选节点，仅当连续 K 次探测彻底断网时才自动切入备用池。
- **细节参数交由用户把控（高级设置项）**：
  - **探测周期（Interval）**：如 30s / 1m / 5m / 仅在网络异常时被动触发。
  - **判定失败阈值**：超时时间（如 3000ms）与连续失败次数（如连续 2 次超时才判定不可用，防止偶发丢包误判）。
  - **切换温差阈值（Tolerance）**：如设为 80ms，仅当新节点比当前节点快 80ms 以上，或当前节点彻底断连时才执行切换。
  - **冷却与熔断恢复（Cooldown）**：被判定不可用的节点进入冷却期（如 10 分钟），期间不参与自动切换，冷却结束后自动进行一次低频复检。

5. 额外推荐的实用功能（提升产品竞争力）

- **按 Wi-Fi / 移动网络自动情景模式（SSID 策略）**：支持设置“连上家里/公司特定 Wi-Fi 时自动直连或关闭代理，切换到蜂窝移动网络或公共 Wi-Fi 时自动开启代理”。
- **二维码扫描与节点分享导出**：支持相机扫码添加节点，以及将单个节点或当前筛选的“高分节点集”一键生成二维码或 SingFlow 专属加密订阅链接分享给其他设备。
- **实时连接面板与分流命中诊断**：参考验收测试中的 realnode_connections.json 与日志能力，在 UI 提供可视化“活跃连接列表”，展示当前每个 App 的目标域名、命中的分流规则、所走的节点及实时上下行速率，方便用户排查“为什么某个软件没走指定国家”。
- **本地 WebDAV 配置云备份**：一键将收藏节点、健康度统计数据、应用分流规则加密备份至坚果云/Nextcloud，换机无缝恢复。
- **快捷磁贴（Quick Settings Tile）与桌面小组件**：支持在 Android 下拉通知栏一键开关代理及快速切换“默认国家分组”。

三、 耗电优化方案（最大化节省电量）

VPN 类应用耗电的核心根源在于**后台高频网络唤醒（Radio State 频繁激活）与UI 线程/日志轮询**。可采取以下针对性措施：

1. **事件驱动替代定时轮询探活（最关键）**
   - **息屏休眠**：监听系统 ACTION_SCREEN_OFF 与 PowerManager.isInteractive。当手机息屏且无高流量传输时，**立即暂停所有后台 urltest 定时探活与 UI 流量速率刷新**；在 ACTION_SCREEN_ON（亮屏）或系统 ConnectivityManager.NetworkCallback 触发网络切换（如 Wi-Fi 切 5G）时，再触发一次快速并发探活。
   - **被动健康检查**：利用代理转发过程中的真实连接错误（如连续 handshake timeout）触发节点切换，而非在后台用高频的心跳包维持探活。
2. **后台日志级别与 IPC 通信节流**

- 生产环境下将 sing-box 日志级别从 debug/info 强制下调为 warn 或 error，避免像测试阶段 realnode_singbox_logs.jsonl 那样高频产生磁盘 I/O 和跨语言（Go <-> Kotlin JNI）字符串拷贝。
- 通知栏速率展示仅在用户下拉通知栏或亮屏时更新，且更新间隔控制在 2000ms 以上。

3. **合理管理 WakeLock 与 CPU 唤醒**
   - 检查 VpnService 实现，除订阅更新或批量测速任务执行期间使用带超时的局部唤醒锁外，日常挂机严禁持有常驻 PARTIAL_WAKE_LOCK，交由系统内核网络栈管理休眠。
4. **订阅更新交给 WorkManager 批处理**
   - 为后台自动更新订阅附加约束条件：setRequiresBatteryNotLow(true)（非低电量）且利用系统维护窗口批处理执行，避免单独唤醒基带。

四、 代码逻辑优化与安装包（APK）体积极限缩减

当前项目包含预编译的 app/libs/libbox.aar（体积通常在 20MB~40MB+）以及构建工作流 .github/workflows/build-libbox.yml，缩小 APK 体积需要从 **Go 内核裁剪、ABI 分包、Android 资源与字节码压缩** 三管齐下：

1. 裁剪 libbox.aar 编译体积（修改 build-libbox.yml）

- **按需编译 Sing-box Tags**：在 .github/workflows/build-libbox.yml 编译 libbox 时，剔除 Android 客户端用不到的服务端模块或冷门组件（例如 with_dhcp、with_lwip（若使用系统栈）、with_tor 等），仅保留核心客户端 Tag：with_quic,with_utls,with_wireguard,with_ech,with_gvisor。
- **开启 Go 编译符号表剥离与路径裁剪**： 在 gomobile / sing-box 编译参数中注入 -ldflags="-s -w -buildid=" 和 -trimpath，可直接将 .so 动态库体积缩减 **25%~35%**。

2. 启用 ABI 架构分包（Splits）与原生库压缩

在 app/build.gradle.kts 中配置 ABI 拆分，避免把 arm64-v8a、armeabi-v7a、x86_64 的 .so 塞进同一个包里（真机主流仅需 arm64-v8a）：

```
android {
    splits {
        abi {
            isEnable = true
            reset()
            include("arm64-v8a", "armeabi-v7a") // 生产环境按架构独立出包
            isUniversalApk = true               // 额外保留一个全架构包供备用
        }
    }
    packaging {
        jniLibs {
            // 开启 Legacy 打包以支持安装时对 libgojni.so 进行高倍率 Deflate/LZMA 压缩
            // 可将 APK 下载体积直接再降 40%~50%
            useLegacyPackaging = true
        }
        resources {
            excludes += setOf(
                "META-INF/*.kotlin_module",
                "META-INF/DEPENDENCIES",
                "META-INF/LICENSE*",
                "DebugProbesKt.bin"
            )
        }
    }
}
```

3. 开启 R8 深度优化与资源缩减

在 app/build.gradle.kts 的 release 构建类型中启用完整 R8 模式：

```
buildTypes {
    release {
        isMinifyEnabled = true
        isShrinkResources = true
        proguardFiles(
            getDefaultProguardFile("proguard-android-optimize.txt"),
            "proguard-rules.pro"
        )
    }
}
```

```
androidResources {
    // 仅保留中文和英文语言包，剔除 AndroidX 库自带的几十种多国语言字符串
    localeFilters += listOf("zh", "zh-rCN", "en")
}
```

4. GeoIP / Geosite 规则集外置与按需热更新

- **不要在** `assets` **中硬编码完整版** `geoip.db` **和** `geosite.db`（两个文件合计往往超过 15MB）。
- **优化方案**：充分利用 `sing-box 1.8+` 的 **Rule-Set (**`.srs`**)** 二进制格式。客户端首次启动或开启“国内分流”时，仅从 CDN 按需下载极其轻量的 `geoip-cn.srs`（约 100KB）和 `geosite-cn.srs`（约 500KB）并缓存在 `context.filesDir`，既保证了规则实时最新，又能为安装包直接减重十余兆。

你是希望先从**修改** `build.gradle.kts` **与** `build-libbox.yml` **进行体积优化**开始，还是先落地**节点健康度评分模型与按应用分流的配置生成器代码**？我可以针对你选定的模块直接提供对应的 Kotlin 实现代码。

User prompt: 我需要的是一份及其详细的设计方案报告可以直接落地开发。另外我也需要提升软件的稳定性。你好好考虑，详细设计每一处细节，力求精细精细再精细，目前只支持中文即可。给我一份详细设计文档。

Response: SingFlow Android 客户端详细工程设计说明书（可直接落地开发版）

本设计文档基于 `SingFlow` 现有代码库（基于 `libbox.aar` / `sing-box` 核心构建，目标适配 Android API 33–36+），围绕**抗封锁能力强化、全链路稳定性加固、批量节点管理与健康度评分、应用级国家分流、可定制容灾切换、极致省电优化及中文单语言包极限瘦身**七大核心工程目标，提供可直接转化为代码的底层落地方案。

一、 系统总体架构与代码重构规范

为消除业务逻辑耦合、减小冗余代码体积并提升并发稳定性，客户端采用 **Clean Architecture + MVI（Model-View-Intent）** 单向数据流架构，按职责划分为四层：

```
com.yangche.singflow
├── core/
│   ├── box/                # libbox.aar JNI 桥接、PlatformInterface 实现、生命周期与内核守护
│   ├── config/             # Sing-box JSON 配置编译器（抗封锁、国家分组、Per-App 路由生成）
│   ├── probe/              # 批量并发探活引擎、GeoIP 落地国家检测器、被动异常熔断器
│   ├── scorer/             # 节点健康度数学评分引擎与标签状态机
│   ├── failover/           # 自动切换容灾控制器（预设策略 + 用户微调参数执行器）
│   └── power/              # 息屏/亮屏状态机、网络切换监听器、动态采样节流器
├── data/
│   ├── local/              # Room 数据库、DAO、Entity、DataStore 偏好设置
│   ├── parser/             # 多协议 URI / Base64 / Clash YAML / Sing-box JSON 批量解析器
│   └── repository/         # 节点仓库、订阅更新 Worker、应用分流规则仓库
├── domain/
│   ├── model/              # 纯 Kotlin 领域模型（不可变数据类，启用 @Immutable 供 Compose 优化）
│   └── usecase/            # 业务用例层
└── ui/
    ├── home/              # 主页（一键连接、实时速率曲线、累计流量、当前连接节点卡片）
    ├── nodes/             # 左滑节点页（国家分组折叠、健康分标签、收藏置顶、批量测速/地区检测）
    └── settings/          # 右滑设置页（应用级国家分流器、自动切换策略配置、订阅管理、抗封锁开关）
```

代码逻辑精简准则：

1. **统一网络与 JSON 栈**：废弃体积庞大的 `Retrofit + OkHttp + Gson/Jackson` 组合，全局统一使用 **Kotlinx Serialization (**`kotlinx-serialization-json`**)** 处理 JSON 序列化与配置生成，订阅拉取与轻量 HTTP 请求直接复用 `HttpURLConnection` 或内置 `libbox` HTTP 客户端，直接削减约 **1.8MB** 第三方库字节码。
2. **协程作用域收敛**：严禁使用 `GlobalScope`。VPN 核心服务绑定专用的 `SupervisorJob() + Dispatchers.IO` 作用域，UI 层严格使用 `viewModelScope` 与 `repeatOnLifecycle(Lifecycle.State.STARTED)`。

二、 代理抗封锁能力与防 DNS 逃逸详细设计

针对验收日志中曾出现的连接超时（`e2e_api36_attempt1_egress_timeout.log`）与 DNS 逃逸问题（`e2e_api36_realnode_attempt1_dns_escape.log`），在配置编译器 `SingBoxConfigBuilder` 中实施以下四层抗封锁加固。

2.1 抗封锁协议与传输层伪装矩阵

特性模块	sing-box 配置字段映射	默认策略与抗封锁原理
uTLS 指纹伪装	`outbound.tls.utls.enabled = true` `outbound.tls.utls.fingerprint = "chrome"`	消除 Go 原生 `crypto/tls` 握手的 JA3/JA4 特征，支持用户在 `chrome`、`firefox`、`safari`、`edge`、`random` 间切换。
TLS 分片 (Fragment)	`route.rules[].tls_fragment = true` `tls_fragment_fallback_delay = "300ms"`	将 TLS ClientHello 握手包在 TCP 层切分为多个微小片段发送，破坏 DPI（深度包检测）设备对明文 SNI 的特征匹配。
多路复用与长度填充	`outbound.multiplex.enabled = true` `protocol = "h2mux",padding = true`	启用连接复用减少高频建连的握手暴露面；强制开启 `padding` 随机填充数据包长度，抵抗基于包长度分布的流量分析。
Reality 偷跑伪装	`outbound.tls.reality.enabled = true` `public_key,short_id`	针对 VLESS 协议，消除服务端自签证书特征，直接复用目标大厂站点的真实 TLS 1.3 证书与旁路握手。

QUIC 抗丢包与端口跳跃	`type = "hysteria2" / "tuic"`
	`server_ports = ["20000:50000"],`
	`hop_interval = "30s"`
		针对 UDP QoS 限流与晚高峰丢包，每 30 秒在指定端口段动态切换本地/远端通信端口，阻断基于五元组的流控封锁。

2.2 杜绝 DNS 逃逸的双栈安全 DNS 架构（Anti-DNS Escape）

为彻底根治 `dns_escape` 问题，客户端**全面禁止任何明文 UDP/TCP 53 端口 DNS 请求直接出网**，采用 **FakeIP + 加密远端 DoH + 严格路由劫持**：

```
{
  "dns": {
    "servers": [
      {
        "tag": "dns-remote",
        "address": "https://1.1.1.1/dns-query",
        "address_resolver": "dns-bootstrap",
        "strategy": "prefer_ipv4",
        "detour": "proxy-default"
      },
      {
        "tag": "dns-direct",
        "address": "https://223.5.5.5/dns-query",
        "address_resolver": "dns-bootstrap",
        "strategy": "prefer_ipv4",
        "detour": "direct-out"
      },
      {
        "tag": "dns-bootstrap",
        "address": "223.5.5.5",
        "detour": "direct-out"
      },
      {
        "tag": "dns-fakeip",
        "address": "fakeip"
      }
    ],
    "rules": [
      {
        "outbound": "any",
        "server": "dns-direct"
      },
      {
        "rule_set": ["geosite-cn"],
        "server": "dns-direct"
      },
      {
        "query_type": ["A", "AAAA"],
        "server": "dns-fakeip",
```

```
        "rewrite_ttl": 1
      }
    ],
    "fakeip": {
      "enabled": true,
      "inet4_range": "198.18.0.0/15",
      "inet6_range": "fc00::/18"
    },
    "final": "dns-remote",
    "independent_cache": true
  }
}
```

- **防逃逸核心机制：**

  1. `"outbound": "any", "server": "dns-direct"`：代理节点自身的域名（Server Address）解析强制走国内加密 DoH，避免鸡生蛋死锁导致的 `egress_timeout`。
  2. TUN 网卡层强制拦截所有 `port = 53` 及 `protocol = "dns"` 的流量并交由内部 `dns-out` 处理，防止国内 App 硬编码 `8.8.8.8:53` 绕过系统 DNS 造成 DNS 泄漏。
  3. 境外 A/AAAA 请求立即返回 `198.18.0.0/15` 网段的 FakeIP，将真实域名随代理隧道发往远端节点解析，本地零暴露、零等待。

三、 数据层详细设计（Room 数据库 Schema）

数据库版本采用 Room 持久化管理，所有表字段均针对高频排序、国家分组、健康度计算及应用分流进行了索引优化。

3.1 节点实体表 (`nodes`)

```
@Entity(
    tableName = "nodes",
    indices = [
        Index(value = ["configHash"], unique = true),
        Index(value = ["countryCode", "healthScore"]),
        Index(value = ["isFavorite", "healthScore"]),
        Index(value = ["subscriptionId"])
    ]
)
data class NodeEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val configHash: String,              // SHA-256(protocol + server + port +credential + transport) 用于幂等去重
    val subscriptionId: Long?,           // 所属订阅 ID，手动导入为 null
    val rawUri: String,                  // 原始分享链接或标准化的 Outbound JSON 字符串
    val displayName: String,             // URLDecode 规范化后的节点名称
    val protocol: String,                // vless, vmess, trojan, hysteria2, tuic, shadowsocks
    val serverHost: String,
    val serverPort: Int,

    // 地区与分组属性
    val countryCode: String = "UN",      // ISO 3166-1 alpha-2 国家代码（如 US, JP, SG, HK, UN=未知）
    val countryNameCn: String = "未知地区",// 中文国家名（如 美国、日本、新加坡）
    val egressIp: String? = null,        // 实测落地出口 IP

    // 探活与健康度统计属性
    val probeTotalCount: Int = 0,        // 累计探活总次数 N
    val probeSuccessCount: Int = 0,      // 累计探活成功次数 S
    val consecutiveFailures: Int = 0,    // 连续失败次数 F_streak（一旦成功即清零）
    val lastLatencyMs: Int = -1,         // 最近一次实测 RTT 延迟（ms，-1 表示未测或超时）
    val emaLatencyMs: Float = 0f,        // 指数移动平均延迟 L_ema
    val latencyJitterMs: Float = 0f,     // 延迟抖动方差评估
    val healthScore: Int = 50,           // 综合健康分（0 - 100，默认初始分 50）
    val healthTag: String = "未检测",     // 动态标签：极佳 / 健康 / 一般 / 异常
    val lastProbeTimeMs: Long = 0L,      // 上次探活时间戳
    val cooldownUntilMs: Long = 0L,      // 故障熔断冷却截止时间戳

    // 用户交互属性
    val isFavorite: Boolean = false,     // 是否收藏
    val cumulativeRxBytes: Long = 0L,    // 该节点累计下行流量
    val cumulativeTxBytes: Long = 0L     // 该节点累计上行流量
)
```

3.2 订阅源表 (`subscriptions`)

```
@Entity(tableName = "subscriptions")
data class SubscriptionEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val name: String,
```

```
    val url: String,
    val userAgent: String = "SingFlow/1.0 (sing-box; Android)",
    val autoUpdateEnabled: Boolean = true,
    val updateIntervalHours: Int = 12,
    val uploadBytes: Long = 0L,          // 解析自响应头 subscription-userinfo
    val downloadBytes: Long = 0L,
    val totalBytes: Long = 0L,
    val expireTimestampSec: Long = 0L,
    val lastUpdatedMs: Long = 0L
)
```

3.3 应用级国家分流规则表 (app_route_rules)

```
@Entity(
    tableName = "app_route_rules",
    indices = [Index(value = ["packageName"], unique = true)]
)
data class AppRouteRuleEntity(
    @PrimaryKey val packageName: String, // 如 "com.google.android.youtube"
    val appName: String,                 // 缓存应用中文名，供列表秒开展示
    val targetMode: String,              // "COUNTRY"（走指定国家）, "DIRECT"（直连）, "BLOCK"（断网）, "NODE"（固定单节点）
    val targetValue: String,             // 当 targetMode="COUNTRY" 时存国家码如 "US"、"JP"；为 "NODE" 时存 configHash
    val updatedAtMs: Long = System.currentTimeMillis()
)
```

四、 核心业务引擎详细设计

4.1 批量节点导入与编码修复引擎 (NodeImportParser)

针对验收测试记录过的节点名称编码异常（e2e_api36_realnode_attempt2_node_name_encoding.log），导入器执行严格的清洗管线：

1. **多源格式嗅探**：
   - 输入文本先去除首尾空白，若不含 :// 且符合 Base64 特征（含 URL-Safe Base64 _ 与 -，自动补齐 = Padding），先执行 Base64 解码再按 \r?\n 拆分为行列表。
   - 若以 proxies: 开头，走轻量级 Clash YAML 节点字段提取器；若以 { 开头且包含 "outbounds"，直接走 sing-box JSON 解析器。
2. **节点名称编码规范化（彻底解决乱码与特殊字符崩溃）**：

```
fun normalizeNodeName(rawFragment: String?, fallbackHost: String, port: Int): String {
    if (rawFragment.isNullOrBlank()) return "$fallbackHost:$port"
    return try {
        // 处理 + 号被误当作空格的问题，兼容多次 URL 编码及非标准 UTF-8 字节序列
        val safeRaw = rawFragment.replace("+", "%2B")
        val decoded = URLDecoder.decode(safeRaw, StandardCharsets.UTF_8.name()).trim()
        // 过滤会导致 JSON 序列化或日志解析异常的不可见控制字符
        decoded.replace(Regex("[\\p{Cntrl}&&[^\r\n\t]]"), "").ifEmpty { "$fallbackHost:$port" }
    } catch (e: IllegalArgumentException) {
        rawFragment.trim()
    }
}
```

3. **快速国家预判与幂等合并**：
   - **正则预匹配**：节点入库瞬间，先通过名称中的国旗 Emoji（如 US、JP、SG）或关键字（美国|US|United States、日本|JP|东京|大阪、新加坡|SG|狮城、香港|HK、台湾|TW、韩国|KR、英国|UK、德国|DE）赋予初始 countryCode，保证未执行网络检测前即可按国家初步分组。
   - **去重与状态保留**：通过 configHash 执行 Room @Upsert。当订阅更新时，若 configHash 已存在，仅更新 rawUri 与 displayName，**完整保留用户的** isFavorite、**历史探活次数** probeTotalCount、**健康分** healthScore **与实测** countryCode。

4.2 批量节点探活与批量落地地区检测引擎 (BatchProbeEngine)

若一次性对上百个节点同时发起无限制的网络连接，会导致 Android 系统的文件描述符（FD）耗尽并引发路由器 NAT 表溢出（造成假性超时）。因此采用**有界协程信号量 + 临时轻量级内存沙盒探活**机制：

1. **并发度控制**：
   - 使用 Semaphore(permits = 12) 严格限制同时处于握手状态的节点数不超过 12 个。
   - 单个节点探活硬超时设为 3500ms（通过 withTimeoutOrNull(3500L) 强制包裹）。
2. **两阶段合一探测（探活 + 地区检测）**：

- **模式 A（仅快速探活）**：通过代理节点向 http://cp.cloudflare.com/generate_204 或 https://www.gstatic.com/generate_204 发起 HEAD/GET 请求。注意：**计时起点为发起代理隧道连接前，终点为收到 HTTP 204 响应头**（即 True URLTest RTT，而非仅测本地到中转机的 TCP Ping），防止中转节点“入口通、落地死”产生虚假连通性。
- **模式 B（探活 + 批量地区检测）**：直接通过被测节点代理请求 https://www.cloudflare.com/cdn-cgi/trace（响应体仅 150 字节纯文本，极省流量且全球 Anycast 节点响应极快）。
- 解析返回的键值对：

  ```
  ip=104.28.x.x
  loc=JP
  ```

  一次性同时获取**真实 RTT 延迟、落地出口 IP (ip) 以及权威落地国家代码 (loc)**，将 loc 映射为中文国家名（如 JP -> 日本）后写入 NodeEntity。若目标节点屏蔽 Cloudflare，自动降级请求 https://api.ip.sb/geoip 或 http://ip-api.com/json/?fields=status,countryCode,query。

4.3 节点健康度评分算法与动态打标签机制 (NodeHealthScorer)

每次探活（无论是用户手动批量探活，还是后台自动切换触发的探测）完成后，立即调用评分引擎更新该节点的统计指标与标签。

1. 核心指标更新公式

设节点历史总检测次数为 N，历史成功次数为 S，连续失败次数为 F_{streak}，历史指数移动平均延迟为 L_{ema}，历史抖动为 J_{ema}。 当本次探测结果为**成功**，且测得延迟为 L_{cur}（单位：ms）时：

- $N' = N + 1, \quad S' = S + 1, \quad F'_{\text{streak}} = 0$
- 指数移动平均延迟（平滑系数 $\alpha = 0.3$）：

$$L'_{\text{ema}} = \begin{cases} L_{\text{cur}}, & \text{若 } L_{\text{ema}} \le 0 \\ 0.7 \times L_{\text{ema}} + 0.3 \times L_{\text{cur}}, & \text{若 } L_{\text{ema}} > 0 \end{cases}$$

- 延迟抖动更新：

$$J'_{\text{ema}} = 0.7 \times J_{\text{ema}} + 0.3 \times |L_{\text{cur}} - L'_{\text{ema}}|$$

当本次探测结果为失败（超时或连接重置）时：

- $N' = N + 1, \quad S' = S, \quad F'_{\text{streak}} = F_{\text{streak}} + 1$

2. 综合健康度评分公式（0–100 分）

综合健康度 Score 由**贝叶斯平滑成功率得分（权重 55 分）、低延迟得分（权重 30 分）、稳定性抗抖动得分（权重 10 分）、历史样本可信度奖励（权重 5 分）减去连续失败重罚项**构成：

1. **拉普拉斯平滑成功率** R_{smooth}（解决仅测 1 次成功就虚高满分的问题）：

$$R_{\text{smooth}} = \frac{S' + 1}{N' + 2}$$

2. **延迟得分** S_{latency}（以 80ms 以内为满分，1200ms 以上为 0 分）：

$$S_{\text{latency}} = 30 \times \text{clamp}\left(\frac{1200 - L'_{\text{ema}}}{1200 - 80},\ 0.0,\ 1.0\right)$$

3. **抗抖动得分** S_{jitter}：

$$S_{\text{jitter}} = 10 \times \text{clamp}\left(1.0 - \frac{J'_{\text{ema}}}{300},\ 0.0,\ 1.0\right)$$

4. **样本置信度分** S_{conf}（鼓励经受过多次检测考验的老节点，检测满 20 次拿满 5 分）：

$$S_{\text{conf}} = 5 \times \min\left(1.0,\ \frac{\ln(N' + 1)}{\ln(21)}\right)$$

5. **连续失败惩罚项** P_{fail}（一旦节点刚挂掉，分数必须立刻大幅下挫以触发自动切换）：

$$P_{\text{fail}} = F'_{\text{streak}} \times 18$$

最终得分汇总：

$$\text{Score} = \text{round}\Big(\text{clamp}\big(55 \times R_{\text{smooth}} + S_{\text{latency}} + S_{\text{jitter}} + S_{\text{conf}} - P_{\text{fail}},\ 0,\ 100\big)\Big)$$

3. 节点健康度标签分级表

分数区间与条件	标签名称 (healthTag)	UI 视觉徽章颜色	调度器赋予的优先级
$\text{Score} \geq 85$ 且 $F'_{\text{streak}} = 0$	**极佳 · 专线级**	翠绿色 (#10B981)	一级主力池（自动切换首选）
$70 \leq \text{Score} < 85$ 且 $F'_{\text{streak}} = 0$	**健康 · 稳定**	蔚蓝色 (#3B82F6)	二级主力池（常规候补）
$50 \leq \text{Score} < 70$ 且 $F'_{\text{streak}} \leq 1$	**一般 · 可用**	琥珀色 (#F59E0B)	三级保底池（仅当主力全挂时启用）
$\text{Score} < 50$ 或 $F'_{\text{streak}} \geq 2$	**异常 · 故障**	珊瑚红 (#EF4444)	自动剔除出活跃自动切换池，进入冷却
$N' = 0$	**未检测**	中性灰 (#9CA3AF)	仅在手动选择或全量待测时使用

4.4 节点按国家分组与应用级国家分流 (CountryRoutingCompiler)

此模块实现用户诉求：**“节点按国家分组；自定义给某些软件走哪些国家，另一些走另外的国家，未设置的走默认国家”**。

1. 节点按国家自动聚合出站组（Outbounds 构建逻辑）

当用户点击连接时，SingBoxConfigBuilder 从数据库读取所有非残损节点，按 countryCode 分组：

- 为每个存在的国家（如 US、JP、SG、HK）生成两个嵌套的 sing-box 出站组：
  - auto-{Country}（类型为 urltest，包含该国下健康分排名前 10 的健康节点，负责该国组内的自动测速优选）。
  - group-{Country}（类型为 selector，成员第一位为 auto-{Country}，后续为该国按健康分降序排列的具体节点，允许用户在特定国家内既可自动优选也可手动指定某个具体节点）。
- 生成一个全局默认出站组 proxy-default（指向用户设定的**默认国家出站组**，如 group-SG；若用户将默认设为“全局智能最优”，则指向汇总了全网最高健康分节点的 group-GLOBAL）。

2. 应用级分流规则编译（Route Rules 构建逻辑）

从 app_route_rules 表中读取用户的分流配置，按目标归类合并 package_name 数组（合并同一国家的包名以大幅减少规则条数，提升内核匹配性能）：

```
{
  "route": {
    "auto_detect_interface": true,
    "override_android_vpn": true,
    "rules": [
      {
        "protocol": "dns",
        "outbound": "dns-out"
      },
      {
        "ip_is_private": true,
        "outbound": "direct-out"
      },
      {
        "package_name": [
          "com.tencent.mm",
          "com.eg.android.AlipayGphone"
        ],
        "outbound": "direct-out"
      },
      {
        "package_name": [
          "com.openai.chatgpt",
          "com.netflix.mediaclient",
          "com.HBO.max"
        ],
        "outbound": "group-US"
      },
```

```
      {
        "package_name": [
          "jp.naver.line.android",
          "com.dmm.app.movieplayer"
        ],
        "outbound": "group-JP"
      },
      {
        "package_name": [
          "org.telegram.messenger",
          "com.google.android.youtube"
        ],
        "outbound": "group-SG"
      },
      {
        "rule_set": ["geosite-cn", "geoip-cn"],
        "outbound": "direct-out"
      }
    ],
    "final": "group-HK"
  }
}
```

- **兜底逻辑**：上方示例中，微信/支付宝强制走直连；ChatGPT/Netflix 强制走美国组 (`group-US`)；Line 强制走日本组 (`group-JP`)；Telegram/YouTube 强制走新加坡组 (`group-SG`)；**其余所有未单独设置的应用，在经过国内域名/IP 直连过滤后，全部自动落入 `"final": "group-HK"`（用户设定的默认国家：香港）**。
- **某国家节点全军覆没时的降级保护**：若用户指定某软件走 `group-JP`，但当前订阅中没有日本节点或日本节点全部不可用，配置编译器自动将 `group-JP` 的 fallback 成员指向 `proxy-default`（默认国家），防止该软件直接断网。

4.5 节点不可用自动切换方案（预设大方向 + 细节自定义）

为彻底解决传统客户端“要么死守断网节点不切、要么稍微抖动就疯狂乱切导致长连接断开”的痛点，设计“四大预设方向 + 七项微调参数”的双层状态机。

1. 四大预设切换方向（Macro Presets）

预设方案名称	适用场景	核心调度与降级方向逻辑
方案 A：国家锁定 · 健康优先（默认推荐）	刷流媒体、AI 助手（防跨国风控封号）	当某节点不可用时，**严格限制在同一国家分组内**寻找替代节点，优先切换至同国家内 `healthScore` 最高且未处于冷却期的节点；仅当该国所有节点连续探测全部阵亡时，才按用户设定的“备用国家顺序（如 SG -> JP -> US）”跨国容灾。
方案 B：收藏池优先 · 极速容灾	拥有自建优质节点或精选节点的用户	优先在已打星标的 `isFavorite = true` 节点池中按综合健康分与实时 RTT 切换；若收藏池全部不可用，再自动降级扩展至全量订阅中 `healthScore >= 75` 的高分池。
方案 C：全域极速 · 低延迟竞技	游戏加速、高频网页秒开	打破国家限制（已单独绑定固定国家的 App 除外），在全网所有健康节点中实时追踪 RTT 最低的节点；一旦当前节点延迟恶化或丢包，立刻无感切往全网最快节点。
方案 D：稳健防抖 · 断网保底（长连接模式）	挂机下载、SSH 终端、金融行情	平时锁定当前连接节点，**完全关闭因延迟波动触发的择优切换**；仅当当前节点彻底断连（连续 K 次探测全部超时）时，才执行一次性故障转移。

2. 用户可把控的七项细节参数（Micro Parameters）

在“自动切换高级设置”面板中，将以下参数开放给用户通过滑块或开关精准微调（持久化在 `DataStore`）：

```
@Serializable
data class FailoverPolicyConfig(
    val presetMode: FailoverPreset = FailoverPreset.COUNTRY_LOCKED_HEALTH,
    // 1. 触发模式：被动异常触发 + 亮屏周期巡检
    val enablePassiveTrigger: Boolean = true,      // 当底层的连网请求出现连续握手错误时立即触发探活
    val activeCheckIntervalSec: Int = 180,         // 亮屏状态下的主动探活间隔（可选 30s / 60s / 180s / 300s / 关闭）

    // 2. 故障判定严格度（防误判）
    val singleProbeTimeoutMs: Int = 3000,          // 单次探活超时判定阈值（1000ms - 5000ms）
    val failConfirmRetries: Int = 2,               // 连续失败确认次数（1 - 5次，默认连续2次失败才定性为不可用）
    val maxAcceptableLatencyMs: Int = 800,         // 慢速熔断阈值：即便未断连，延迟若连续超过 800ms 也视为不可用

    // 3. 切换防抖与冷却控制（防乒乓效应）
    val switchHysteresisScoreDiff: Int = 12,       // 择优切换健康分门槛：候选节点比当前节点高出至少 12 分才切换
```

```
    val switchLatencyToleranceMs: Int = 80,        // 择优切换延迟温差：候选节点比当前节点快至少 80ms 才切换
    val deadNodeCooldownMinutes: Int = 10,         // 故障节点小黑屋冷却时长（1 - 60分钟，冷却期内不参与自动切换）

    // 4. 跨国兜底策略
    val allowCrossCountryFallback: Boolean = true,// 同国家节点全灭时，是否允许切往默认国家/备用国家
    val fallbackCountryOrder: List<String> = listOf("SG", "JP", "HK", "US"),
    val notifyOnSwitch: Boolean = false            // 自动切换发生时是否弹出 Toast / 静默通知提示
)
```

3. 自动切换执行状态机（核心流程）

1. **故障捕获**：当 ActiveHealthMonitor 检测到当前活跃节点探活超时，立即间隔 500ms 发起第 $2...K$ 次快速复核（failConfirmRetries）。
2. **本机断网甄别（极其关键，防误杀）**：若复核依然失败，在判定代理节点阵亡前，**先通过** direct-out **直连通道对国内高可用地址（**http://connect.rom.miui.com/generate_204 **或** 223.5.5.5**）发起一次** 1500ms **探测**：
   - 若直连探测也失败 → 说明是用户手机自身没网（进电梯、地下室、Wi-Fi 断网），**保持当前节点不变，不扣减节点健康分，不触发乱切**，等待系统网络恢复。
   - 若直连探测成功、代理探测失败 → 坐实当前代理节点不可用！
3. **熔断与优选切换**：
   - 立即更新当前故障节点的健康分（F_{streak} 累加，健康分骤降），并将其 cooldownUntilMs 设为 now + deadNodeCooldownMinutes * 60_000。
   - 根据当前选定的 FailoverPreset 从数据库筛选出未冷却的候选节点前 3 名，并发执行一次快速探活，选出首个响应且综合分最高的节点。
   - 调用 libbox 的 ClashMode / Selector.selectOutbound(tag) 接口，**在毫秒级内直接内存切换出站指针，无需断开和重建 Android** VpnService **TUN 网卡**，实现真正的零断流无感切换。

4.6 补充实用功能设计（提升日常使用体验）

1. **实时分流命中与连接诊断器（Connection Inspector）**
   - 复用 sing-box Clash API 兼容层或 CommandClient 连接追踪能力（类似仓库验收中的 realnode_connections.json），在 App 内提供“连接诊断”抽屉：实时展示 应用图标 + 应用名称、目标域名/IP、命中的分流规则（如：走日本组 group-JP）、实际承载节点名 以及 实时吞吐速率，并提供“一键将该应用改为走某国家”的快捷操作。
2. **网络环境智能自动化（Wi-Fi SSID / 移动网络联动）**
   - 支持配置受信任的家庭/公司 Wi-Fi 列表：连上指定 Wi-Fi 时自动暂停代理或切换为“仅境外分流模式”；离开 Wi-Fi 切入 5G 移动网络或连上公共陌生 Wi-Fi 时，自动拉起 VPN 保护。
3. **高分节点集一键导出与二维码分享**
   - 支持将“收藏节点”或“当前健康分 ≥ 80 的节点”一键打包导出为 Base64 订阅文本、Sing-box JSON 配置文件或生成二维码，方便多台手机/电脑间同步高质量节点池。
4. **本地加密备份与 WebDAV 同步**
   - 将订阅源、收藏状态、历史健康度统计、应用级国家分流表打包为单个 Zstd/Gzip 压缩 JSON，支持本地文件导入导出及坚果云/Nextcloud WebDAV 一键同步。
5. **系统状态栏快捷磁贴（**TileService**）**
   - 提供下拉控制中心磁贴，未连接时点击一键启动并连接默认国家最优节点；已连接时副标题直接显示“当前国家 · 节点延迟（如 新加坡 · 42ms）”。

五、 全方位软件稳定性加固工程（Stability Engineering）

为确保 SingFlow 7×24 小时长期挂载不闪退、不断流、不假死，针对 Android VpnService 与 Go 语言 libbox.aar 跨层调用实施六项硬核加固。

5.1 解决切换网络后假死断流（修复 egress_timeout

)

Android 手机在 Wi-Fi 与蜂窝数据（5G/4G）切换、或者双卡切换时，底层物理网卡（wlan0 <-> rmnet_data0）发生变更，若 sing-box 仍绑定在旧网卡的 Socket 上，所有新连接都会陷入 egress_timeout。

- **加固实现：** 在 SingFlowVpnService 中注册 ConnectivityManager.NetworkCallback，并准确实现 libbox.PlatformInterface 的 autoDetectInterfaceControl(fd: Int) 与 openTun：

```
val networkRequest = NetworkRequest.Builder()
    .addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
    .addCapability(NetworkCapabilities.NET_CAPABILITY_NOT_VPN)
    .build()

val networkCallback = object : ConnectivityManager.NetworkCallback() {
    override fun onAvailable(network: Network) {
        // 1. 立即通知 Android 系统将此物理网络设为 VPN 的底层承载网络
        setUnderlyingNetworks(arrayOf(network))
        // 2. 防抖 300ms 后通知 libbox 刷新连接池并重测当前活跃节点
        serviceScope.launch {
            delay(300)
            boxService?.resetNetwork()
            failoverController.triggerImmediateCheck(reason = "NETWORK_CHANGED")
        }
    }
    override fun onLost(network: Network) {
        setUnderlyingNetworks(null)
    }
}
```

- 同时在 autoDetectInterfaceControl(fd: Int) 回调中强制调用 VpnService.protect(fd)，若 protect(fd) 返回 false，立即记录错误并抛出异常终止该次拨号，绝不让流量死循环回流进 TUN 接口。

5.2 消除配置更新时的闪退与僵尸 TUN 接口

- **单实例互斥锁：** 在 BoxServiceManager 中使用 Mutex() 严格串行化 start()、stop()、reload() 操作。防止用户快速连点“连接/断开”按钮导致两个 BoxService 同时竞争同一个 TUN 文件描述符而引发 Native Crash。
- **优先热重载而非重建 TUN：**
  - 当用户仅切换节点或自动容灾切换时，通过 libbox 内置的 Selector 接口在内存态切换出站，耗时 < 5ms。
  - 当用户修改了“应用分流国家规则”时，先尝试调用 libbox 的配置热重载，若需重建 TUN，遵循“先关闭 BoxService -> 关闭 ParcelFileDescriptor -> 延迟 150ms 确认内核释放 -> 重新 Builder().establish()”的严格时序。

5.3 Go Native 内存水位限制与防 OOM 杀后台

Go 运行时的垃圾回收器（GC）默认倾向于占用较多虚拟内存，在并发探活或大流量下载时容易触发 Android LMK（Low Memory Killer）直接杀死 VPN 进程。

- **加固实现：**
  1. 在 sing-box 实验性配置块中开启内存上限约束：

```
"experimental": {
  "cache_file": {
    "enabled": true,
    "path": "cache.db",
    "store_fakeip": true
  }
}
```

  2. 在编译 libbox 或启动初始化时，调用 libbox.setMemoryLimit(true)（或设置环境变量 GOMEMLIMIT=45MiB），将 Go 内核堆内存严格限制在 **45MB** 以内，超出水位立即触发 Go 增量 GC。
  3. 适配 Android 14–16 (API 34–36) 前台服务强校验：在 AndroidManifest.xml 中为 SingFlowVpnService 显式声明 android:foregroundServiceType="specialUse"，并在 Service.onStartCommand 的 **3 秒内**完成 startForeground(NOTIFICATION_ID, notification, ServiceInfo.FOREGROUND_SERVICE_TYPE_SPECIAL_USE) 调用，彻底杜绝 ForegroundServiceDidNotStartInTimeException。

5.4 文件描述符（FD）耗尽防护与配置原子写入

- **FD 上限保护**：在生成的 sing-box 路由与传输层配置中，将 UDP NAT 会话超时 udp_timeout 从默认的 5m 缩短为 "90s"，及时回收王者荣耀、语音通话或 QUIC 产生的闲置 UDP 句柄。
- **配置原子写入**：所有写入本地磁盘的配置文件（如 running_config.json）一律使用 androidx.core.util.AtomicFile，防止写入中途断电或进程被杀产生半截损坏的 JSON 导致下次启动崩溃。

六、 极致耗电优化设计（Power Optimization）

代理软件后台耗电的三大元凶是：**息屏期间的高频定时网络探活（阻止手机基带进入休眠态）、高频 UI 流量图刷新**与**海量内核日志输出**。

6.1 屏幕与运行态感知的三级功耗状态机 (PowerStateController)

通过监听系统广播 Intent.ACTION_SCREEN_ON、Intent.ACTION_SCREEN_OFF、PowerManager.ACTION_POWER_SAVE_MODE_CHANGED 以及 App 前后台生命周期，将客户端划分为三个功耗档位：

运行状态档位	触发条件	速率采样与曲线绘制	自动探活与容灾策略	额外功耗开销
L1：前台活跃态	App 处于前台可见且屏幕点亮	每 1000ms 采样一次上下行速率，渲染 60fps 平滑贝塞尔曲线	正常执行主动探活（如每 180s）+ 被动异常检测	标准（仅占前台时长）
L2：后台亮屏态	用户切到其他 App 使用手机，SingFlow 在后台	**停止 UI 曲线绘制与 Flow 发射**；若开启了通知栏速率显示，采样间隔拉长至 3000ms	停止全量节点轮询，仅对**当前正在使用的活跃出站节点**执行低频健康检查 + 被动异常熔断	极低（< 0.5% / 小时）
L3：息屏深度休眠态	屏幕关闭（ACTION_SCREEN_OFF）满 30 秒，或系统进入省电模式	**彻底挂起所有速率采集协程、停止通知栏刷新**（消除 NotificationManager.notify 引起的系统 SystemUI CPU 唤醒）	**100% 挂起所有主动定时探活任务**，释放基带连接；仅当亮屏瞬间或网络切换时执行一次唤醒复检	接近零耗电（与未挂代理待机持平）

6.2 砍断高频日志与跨语言 JNI 拷贝开销

在验收测试阶段，项目生成了详细的 realnode_singbox_logs.jsonl 日志文件。但在日常使用中，每转发一个请求就通过 JNI 回调将日志字符串传给 Kotlin 层并落盘，会产生巨大的 CPU 唤醒和闪存写放大。

- **省电改造**：
  1. **日常模式默认日志级别设为 "warn" 或 "error"**：仅当用户当前正打开 App 内的“日志排查页”时，才动态开启日志流订阅；一旦用户滑出日志页或退到后台，立即注销日志回调流。
  2. **环形内存缓冲替代磁盘实时追加**：日志页面采用固定容量为 200 条 的内存 ArrayDeque 环形队列，不写磁盘。

七、 安装包（APK）极限瘦身与构建工程配置（仅保留中文）

鉴于当前只需支持**中文**，且项目中包含 app/libs/libbox.aar 及 .github/workflows/build-libbox.yml，通过以下全套工程配置可将单架构 Release APK 体积压缩至极致（相比未裁剪的通用包缩减 **60% 以上**）。

7.1 改造 .github/workflows/build-libbox.yml 裁剪 Go 内核体积

在编译 libbox.aar 时，剔除用不到的服务端模块、调试符号表、DWARF 信息，并仅保留主流真机架构（arm64-v8a）：

```
# 在 .github/workflows/build-libbox.yml 的编译步骤中使用精简 Tags 与极限 ldflags
- name: Build optimized libbox.aar
  run: |
    # 仅保留客户端抗封锁与核心分流必需的编译标签，剔除 with_lwip, with_dhcp, with_tor, with_acme 等冗余模块
    BUILD_TAGS="with_gvisor,with_quic,with_utls,with_wireguard,with_ech,with_clash_api"

    # -s: 移除符号表; -w: 移除 DWARF 调试信息; -buildid=: 清空构建ID; -trimpath: 移除编译机器绝对路径字符串
    gomobile bind -v -androidapi 28 -target android/arm64 \
      -tags "$BUILD_TAGS" \
      -trimpath \
      -ldflags="-s -w -buildid=" \
      -o app/libs/libbox.aar \
      ./experimental/libbox
```

- **优化效果**：仅编译 android/arm64 并加上 -s -w -trimpath 后，libbox.aar 内的 libgojni.so 原始体积直接减少约 **35%**，且彻底消除了 x86/x86_64 模拟器冗余库对正式包的体积占用。

7.2 改造 app/build.gradle.kts（中文单语言过滤 + R8 深度压缩 + So 极限压缩）

```
android {
    namespace = "com.yangche.singflow"
    compileSdk = 36

    defaultConfig {
        applicationId = "com.yangche.singflow"
        minSdk = 33
        targetSdk = 36
        versionCode = 1
        versionName = "1.0.0"

        // 1. 仅保留arm64真机架构（如需兼容极老旧32位设备可额外配置 splits）
        ndk {
            abiFilters += listOf("arm64-v8a")
        }
    }

    // 2. 严格限制仅保留中文资源，彻底剥离 Jetpack / Material3 库自带的 80+ 种外国语言包
    androidResources {
        localeFilters += listOf("zh", "zh-rCN", "zh-rHK", "zh-rTW")
    }

    buildTypes {
        release {
            // 3. 开启 R8 代码混淆、无用类剔除与无用资源物理裁剪
            isMinifyEnabled = true
            isShrinkResources = true
            isDebuggable = false
            isJniDebuggable = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }

    packaging {
        jniLibs {
            // 4. 关键体积优化：开启 Legacy 打包，让 APK 打包器对 libgojni.so 进行 Deflate 高倍率压缩
            // 可将最终生成的 APK 文件大小直接减小 12MB ~ 18MB
            useLegacyPackaging = true
        }
        resources {
            // 5. 剔除所有无用的元数据与协程调试探针文件
            excludes += setOf(
                "DebugProbesKt.bin",
                "kotlin-tooling-metadata.json",
                "META-INF/*.kotlin_module",
                "META-INF/*.version",
                "META-INF/AL2.0",
                "META-INF/LGPL2.1",
                "META-INF/DEPENDENCIES",
                "META-INF/LICENSE*",
                "META-INF/NOTICE*"
            )
        }
    }
}
```

7.3 配套 app/proguard-rules.pro 深度瘦身与防崩规则

在 gradle.properties 中添加 android.enableR8.fullMode=true 启用 R8 激进优化模式，并在 proguard-rules.pro 中配置：

```
# 1. 保护 libbox.aar 的 JNI 桥接类与回调接口不被混淆（极其重要，防止 Release 包启动即闪退）
-keep class io.nekohasekai.libbox.** { *; }
-keep class go.** { *; }

# 2. 保护 Kotlinx Serialization 序列化模型与 Room 数据库实体
-keepattributes *Annotation*, InnerClasses, Signature
-dontnote kotlinx.serialization.AnnotationsKt
-keepclassmembers class com.yangche.singflow.** {
    *** Companion;
}
-keepclasseswithmembers class com.yangche.singflow.** {
```

```
    kotlinx.serialization.KSerializer serializer(...);
}
-keep class * extends androidx.room.RoomDatabase
-keep @androidx.room.Entity class *

# 3. Release 包在字节码层面彻底抹除所有 Log.d / Log.v 调试代码，既减小体积又减少运行时字符串拼接开销
-assumenosideeffects class android.util.Log {
    public static boolean isLoggable(java.lang.String, int);
    public static int v(...);
    public static int d(...);
}

# 4. 允许 R8 重打包与内联短方法以极致压缩 DEX 体积
-repackageclasses 'sf'
-allowaccessmodification
-optimizations !code/allocation/variable
```

7.4 静态资源与分流规则库精简策略

1. **零内置大体积 Geo 数据库**：绝不在 `app/src/main/assets` 中打包传统的 `geoip.db` 和 `geosite.db`（合计约 18MB）。改为首次启动或后台静默拉取 `sing-box` 专属的高压缩二进制规则集 `geoip-cn.srs`（~90KB）与 `geosite-cn.srs`（~480KB），缓存在 `context.noBackupFilesDir`。
2. **全矢量图标（Vector Drawable）**：所有 UI 图标、国旗徽章与状态指示器一律采用精简路径的 SVG `VectorDrawable` 或直接复用系统内置 Emoji 渲染，完全剔除 `mdpi/hdpi/xhdpi/xxhdpi` 下的多套 `.png` 位图资源。

八、 落地开发实施路线与验收清单（Milestones）

建议按照以下 4 个阶段在本地或通过自动化编码 Agent 顺序推进落地：

1. **阶段一：基础底座与体积/稳定性瘦身（Day 1）**
   - 落地第七节的 `app/build.gradle.kts`、`proguard-rules.pro` 及 `.github/workflows/build-libbox.yml` 改造。
   - 落地第五节的 `SingFlowVpnService` 网络切换回调 `NetworkCallback` 与 `AtomicFile` 配置保护，验证 Release `arm64` 包构建成功且无 `egress_timeout`。
2. **阶段二：Room 数据层与批量导入/探活/评分引擎（Day 2）**
   - 创建第三节的 Room 数据库表结构。
   - 实现 `NodeImportParser`（含 URL 编码修复、多协议解析、Hash 去重）与 `SubscriptionWorker`。
   - 实现 `BatchProbeEngine`（12 并发信号量、URLTest + Cloudflare Trace 落地国家识别）与 `NodeHealthScorer`（平滑评分公式与标签分级）。
3. **阶段三：国家分组、应用级国家分流与自动切换容灾（Day 3）**
   - 实现 `SingBoxConfigBuilder`，将节点按 `countryCode` 聚合为 `group-{Country}`，并将 `app_route_rules` 编译为 `route.rules`，注入第二节的抗封锁与防 DNS 逃逸模板。
   - 实现 `FailoverController`，接入四大预设方向与七项用户微调参数，跑通“本机直连甄别 -> 节点熔断冷却 -> Selector 内存态毫秒级切换”闭环。
4. **阶段四：UI 交互对接与三级省电状态机（Day 4）**
   - 对接主页（一键连接、实时速率曲线、当前节点）、左滑节点页（国家折叠列表、收藏置顶、健康度彩色标签、批量操作栏）、右滑设置页（应用分流国家选择器、自动切换微调面板）。
   - 挂载第六节的 `PowerStateController`，完成息屏零轮询与日志节流验收。

User prompt: 生成md文档，我要下载

Response:
